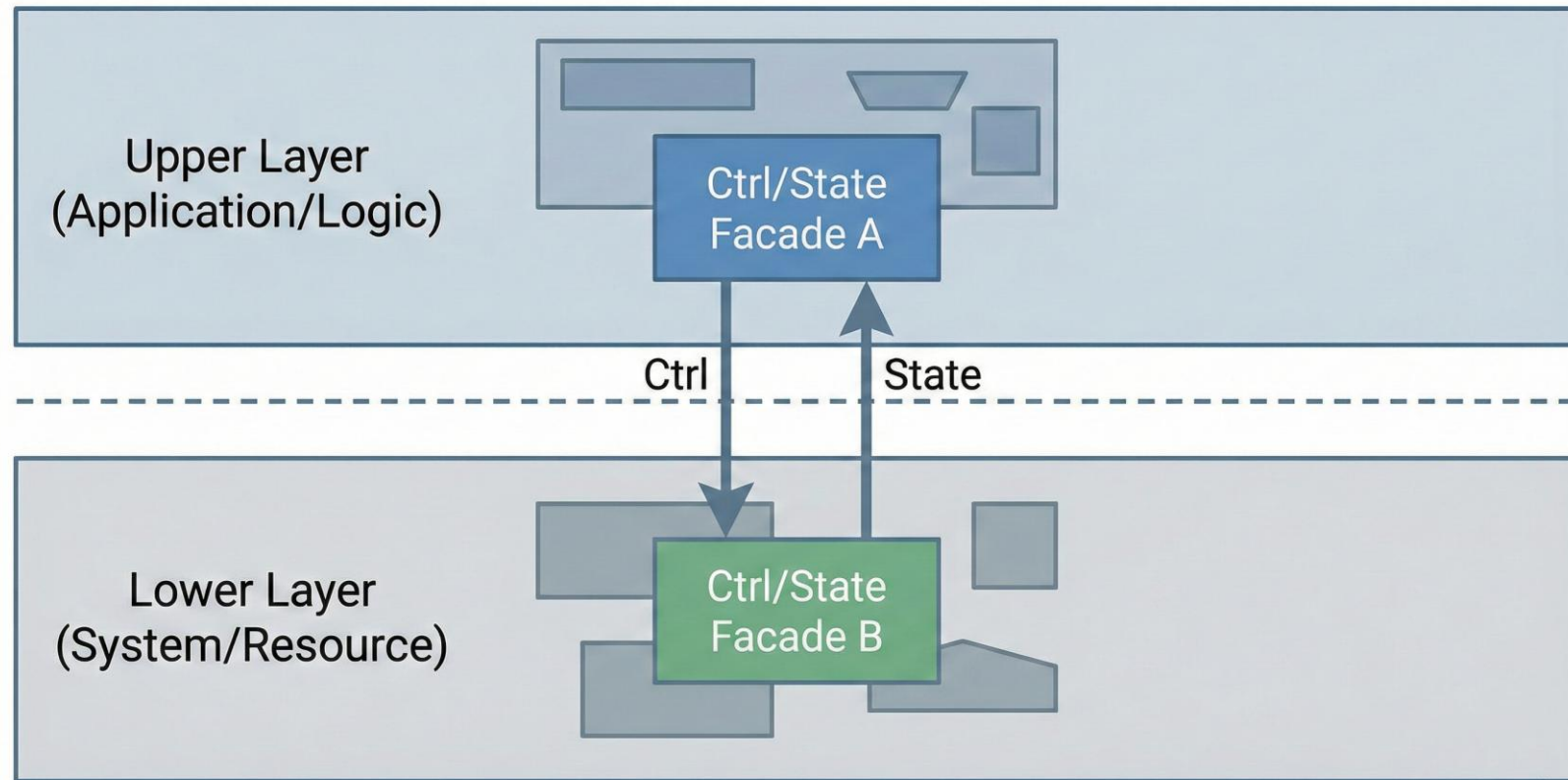
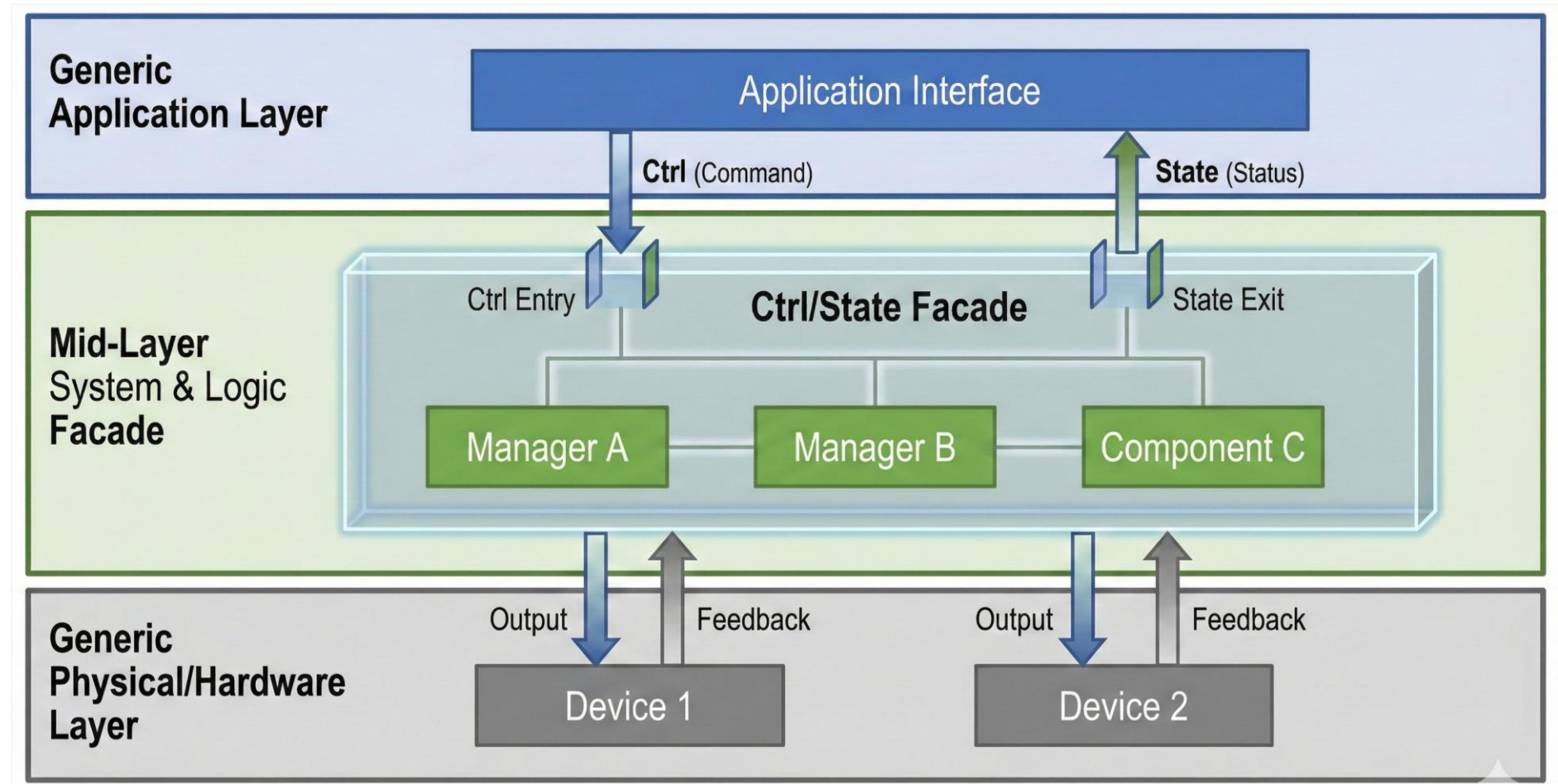




1. Motivation
2. Design



▪ Motivation

- Unified, transport-independent command and feedback structures
- Higher-level logic is shielded from lower-level implementation changes
- Ctrl / State fields can be mapped to cyclic or non-cyclic transports, including fieldbus, ADS, and TCP/IP
- Asynchronous command propagation with cyclic command processing and state ownership.
- Coherent command and state publication

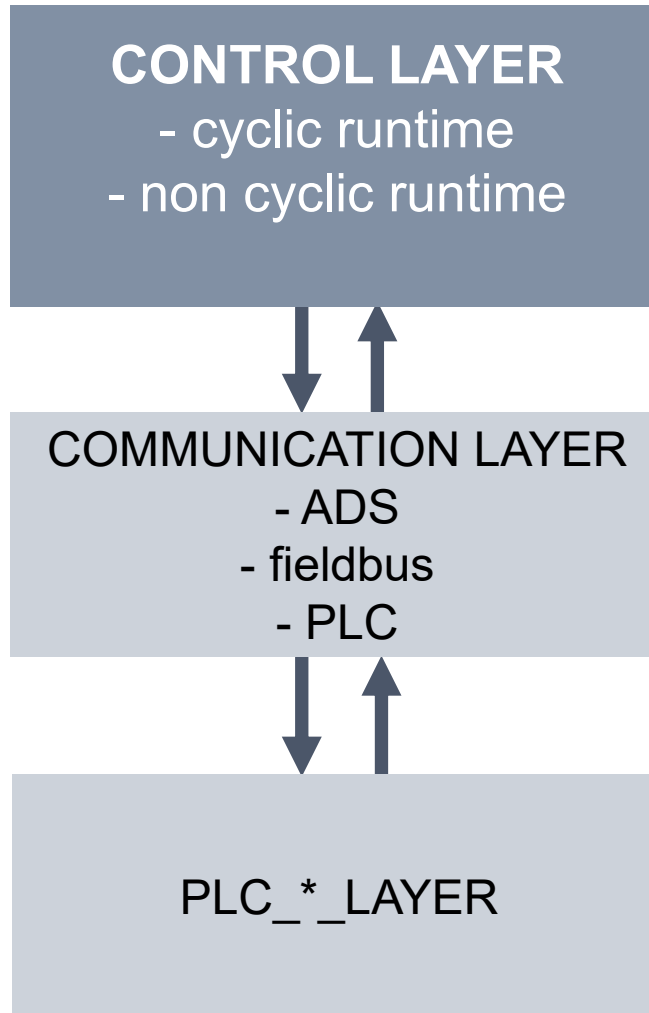
- **Same cycle responsiveness**

- Sequential case execution removes avoidable internal state-transition latency.
- A preallocated, bounded linked structure supports deterministic FIFO handover without array shifting or runtime allocation.
- Commands are available before physical completion is observed.
 - State feedback confirms the asynchronous result.
- Enumerations create readable command, state, and diagnostic narratives.

■ Design

- Designed, built, simulated and proven in practical cyclic-control applications.
 - Guided by SOLID and ACID-inspired principles.
- “Built-in” self-similarity of approach enables fractal design choices.
 - Due to strong **Separation of Concerns** the layered approach is suitable for distributed RTS systems.
- A consistent pattern supports controlled, testable execution and reduces unintended coupling.
 - A facade pattern enables isolated design.
 - State isolation and command execution enable structural inhibition of race conditions.
 - Command ownership is independent of command execution.
- Use of base classes with cyclic wrappers (Ctrl / State) to allow for fractal design.
 - Use of abstract base classes enforces discipline when thinking about permutations of existing extensions.
- Cross-layer communication through narrow interface contracts.
 - Interfaces are contracts: expose only the methods and properties required by the consumer.

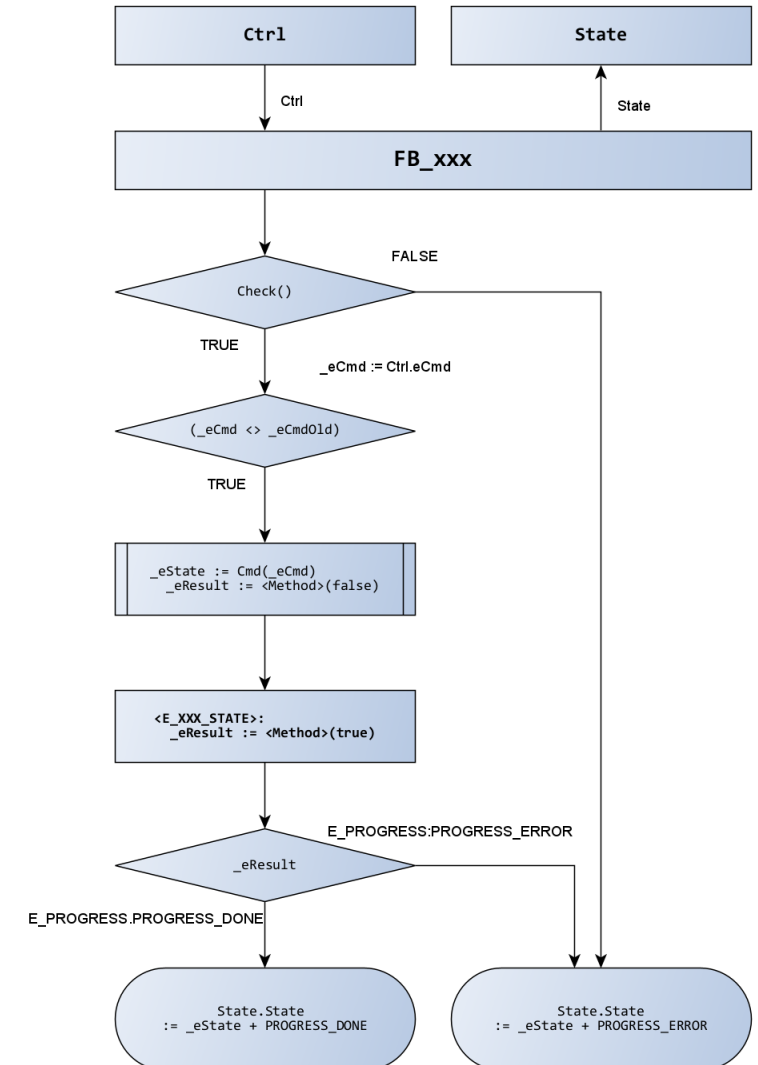
- **Design:**
- Transparency of communication layer
- Code base shall remain independent of control layer
- Configurable Options for specific libraries / TC functions
- Balanced load for configurable options in machine layout
- Stable CPU use for XFC applications



■ Design:

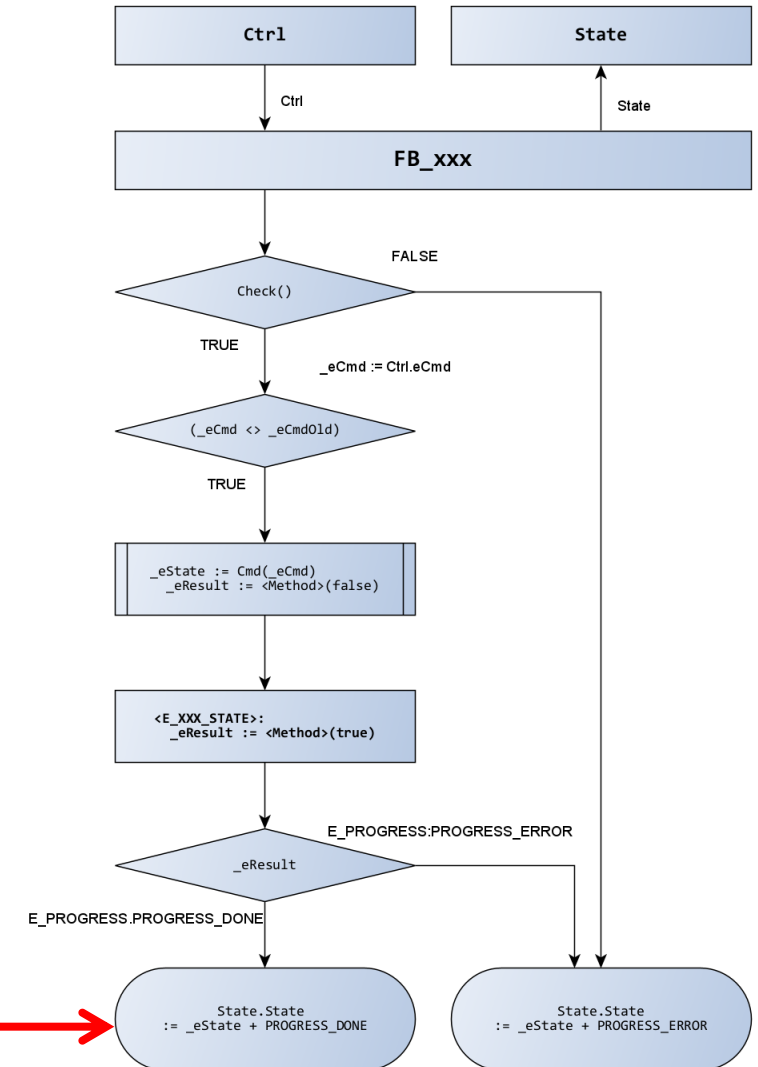
■ Ctrl / State structures

- Establishes unified access. Each Ctrl data field has one dedicated command owner.
- The command owner can simply 'drop' requests into Ctrl
 - The cyclic layer processes them.
- Enables asynchronous communication with PLC_*_LAYER (e.g. C# via ADS, C/C++ via ADS, ADS over MQTT, ...)
- Enables cyclic communication with PLC_*_LAYER since structures can easily be mapped onto any cyclic fieldbus TwinCAT supports (EtherCAT, EAP, CANopen, Profi..., ...)
- The cyclic layer updates State
 - callers can continue asynchronously and later evaluate acknowledgement, completion, abort, timeout, or fault.



■ Design:

- State feedback for cyclic class wrappers
 - **Always** combined with E_PROGRESS
- You can filter your response by a simple modulo division
 - The result may be your entry point for your reaction to State.
- State feedback may be used for cyclic wrappers and methods alike.



■ Design:

– Observable progression of cyclic logic with E_PROGRESS.

```
{attribute 'to_string'}  
TYPE E_PROGRESS :  
(  
    // progress has 2 use cases  
    // 1. as offset to cyclic interface's state  
    //    for the requested command/function  
    //    e.g. State := <Enum equivalent to eCmd> + E_PROGRESS  
    //  
    // 2. as state feedback (result) from (any) method  
    PROGRESS_INVALID,  
    PROGRESS_NOT_EXIST      := 100,  
    PROGRESS_INIT           := 1000,  
    PROGRESS_BUSY           := 2000,  
    PROGRESS_PREPARE        := 3000,  
    PROGRESS_STARTUP        := 4000,  
    PROGRESS_CHECK          := 5000,  
    PROGRESS_OCCUPIED       := 6000,  
    PROGRESS_WORKING        := 7000,  
    PROGRESS_STILL_WORKING  := 8000,  
    PROGRESS_ERROR          := 9000,  
    PROGRESS_DONE           := 10000  
)UINT;  
END_TYPE
```



Shows
progress

Software Design Example:

